

Superseded — see `porygon_review_v3`

This deck is kept for provenance only and is not the current panel review deck. `porygon_review_v3.tex` supersedes it: v3 corrects the confirmatory/pilot evidence-class mislabeling present throughout this file (every “confirmatory” 216-trial or 144-trial result below is pilot evidence, `research_eligible: false`), restores the Lin (1991) and Forrest et al. (1996) foundational citations this file’s bibliography dropped, describes the current broadened POR-DET-004 rule rather than the old fixed dual-use-tool name list, and documents a second, later live XMRig run this file does not include. Read `porygon_review_v3.tex` instead unless you specifically need this file’s historical content.

PORYGON: A Statistical and Rule-Based Runtime Security Framework for Containerized Workloads

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: 75

S.No	Register Number	Student Name
1	CB.EN.U4CSE23504	Ashin Varghese
2	CB.EN.U4CSE23559	Anurup Krishna
3	CB.EN.U4CSE23628	Krishnan S
4	CB.EN.U4CSE23661	Madhav Sreejith

Guide: **Dr Abirami K**, Assistant Professor, Department of CSE

September 13, 2026

Research Question and Contribution

The specific, falsifiable question this project answers

System overview: Porygon ingests Falco eBPF process-execution events and Docker lifecycle events per container image digest, computes the Jensen–Shannon distance between a versioned historical baseline and the current runtime window, and correlates the resulting score against seven deterministic detection rules. A match produces a finding gated by explicit severity/confidence thresholds; no containment action executes without human operator approval.

The unanswered design question: container security tools learn what “normal” process behavior looks like, then flag deviations. But how finely should “normal” be defined? Four real options, in increasing order of specificity:

- ❶ **One-size-fits-all** – a single baseline shared by every container regardless of what image it runs.
- ❷ **Per mutable label** – one baseline per image tag (e.g. `myapp:latest`); the risk is that a tag can be silently repointed to different image content later.
- ❸ **Per exact image** – one baseline per cryptographic image identity, which cannot silently change.
- ❹ **Per exact image + configuration** – the same, but also split by the container’s specific security settings (permissions, capabilities).

No existing tool or published study (Falco, Sysdig, Aqua, or the academic line from Castanhel et al., ISCC 2021, through eBPF-Guard, 2025–2026) measures which of these four actually reduces false alarms. This project does, using 216 real container trials and formal statistical testing rather than a single accuracy figure.

Result: option 3 eliminated false alarms entirely versus option 1 (100% → 0%), without missing any real attack (96/96 caught either way). This was not a fluke – the same result appeared at two different sample sizes ($p = 5.8 \times 10^{-11}$ at 144 trials, $p = 6.7 \times 10^{-16}$ at 216 trials), and got statistically stronger, not weaker, the second time.

Novelty and Positioning Against Prior Work

What is actually new here, stated precisely

What is not new: eBPF-based container process monitoring is an established technique, and Jensen–Shannon divergence is a well-established distance measure, applied to network-traffic intrusion detection as recently as Bouke et al. (arXiv, 2026); both predate this project.

What is new: no located tool or published study – commercial (Falco, Sysdig, Aqua) or academic (Castanhel 2021 through 2025–2026) – measures which of four concrete baseline-scoping granularities (pooled / mutable-tag / exact-digest / digest-plus-configuration) actually reduces false alarms, using controlled trials and significance testing rather than a single accuracy figure. This project runs that comparison and reports both a significant result and a null result from one protocol.

Closest prior work, checked directly:

- Castanhel et al. (IEEE ISCC, 2021) – closest match: container-level, kernel-observed, system-call anomaly detection with no prior knowledge of the workload. Uses ML classifiers over syscall features, not JS-distance; no scoping comparison.
- El Khairi et al. (Cloud Computing Security Workshop, ACM CCSW, 2022) – container-specific HIDS modeling normal syscall context via a graph structure; no baseline-scoping comparison.
- Lin et al., “eBPF-Guard” (*Empirical Software Engineering*, Dec 2025) – same sensor layer, scores via a fine-tuned LLM; single 99.22% accuracy figure, no scoping comparison.
- Li/Yuan/Wu/Cheng (2025) and Xiong/Wu/Jia, “DeepContainer” (2025) – neural container anomaly detectors, single point-estimate accuracy (96.8–99%), no confidence interval or significance test.

All independently verified (DOI resolution or direct publisher fetch); full list in docs/RELATED_WORK_RECENT.md.

System Architecture

Eight containerized services, three network segments

- **falco** – eBPF driver; captures `execve` and lifecycle events at the kernel boundary.
- **collector** – reads Docker daemon events; SQLite outbox with retry delivery.
- **telemetry** – ingests Falco process-execution events, same outbox pattern.
- **backend** – baseline construction, JSD scoring, seven-rule detection engine, response-policy logic.
- **scanner** – Trivy CycloneDX SBOM generation, EPSS and CISA KEV enrichment.
- **responder** – executes approved containment actions only after explicit human approval.
- **gateway** – single ingress point for API and dashboard.
- **postgres** – durable store for events, baselines, scores, findings, incidents, scan results.

Network segregation (`compose.yaml`):

- `porygon_ingress` – external-facing traffic only (gateway).
- `porygon_internal` – inter-service communication.
- `porygon_egress` – outbound-only, granted exclusively to the scanner.

A service outside a segment has no network path to it; the scanner is the only service with outbound internet reachability.

1. Methodology & Solution Design

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Methodological Framework:

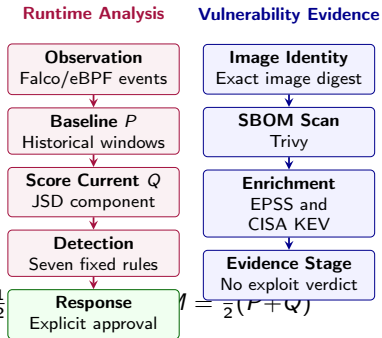
- **Behaviour Baseline:** Historical Docker lifecycle and Falco process events form a versioned baseline for one immutable image digest.
- **Anomaly Scoring:** For each categorical feature, the v1 scorer compares baseline P with a completed runtime window Q using base-2 Jensen-Shannon *distance* (the square root of the Jensen-Shannon *divergence*, bounded in $[0, 1]$):

$$D_{JS}(P, Q) = \sqrt{JSD(P, Q)}, \quad JSD(P, Q) = \frac{1}{2} D_{KL}(P \| M) + \frac{1}{2}$$

Smaller values indicate more similar behaviour.

- **Rule-Based Detection:** Seven deterministic rules produce explainable findings. An incident requires an unsuppressed, incident-eligible finding.

Porygon Workflow



1. Methodology & Solution Design (contd.)

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Implementation Approach and Design Choices:

- **Kernel-Level Monitoring:** Falco's modern eBPF driver captures container process execution without modifying the applications. The Telemetry Adapter ingests those events, while the Docker Collector records lifecycle and image-identity events. Both use bounded SQLite outboxes with retry delivery.
- **Service Integration:** Docker Compose coordinates eight long-running containers. Segregated networks restrict connectivity, and outbound access is limited to the vulnerability scanner.
- **Explainable Analysis:** Base-2 Jensen–Shannon distance provides a bounded, symmetric measure of categorical change. Fixed rules expose the matched evidence, so the implemented path does not depend on a black-box ML classifier.
- **Controlled Response:** Policy thresholds use severity and confidence to determine allowed containment actions. Disruptive execution is disabled by default. When enabled, it still requires explicit human approval.
- **Implementation Verification:** The repository contains 79 backend test functions across baseline, scoring, detection, calibration, response, and vulnerability modules, plus 131 further test functions across the collector, telemetry, responder, and scanner services (210 total).

1. Methodology — The Scoring Mathematics

Rubric: Mathematical Model / Formulation (5 Marks)

Production path – generates every reported number. Base-2 Jensen–Shannon distance between baseline p and observed window q :

$$D_{JS}(p\|q) = \sqrt{\frac{1}{2}D_{KL}(p\|m) + \frac{1}{2}D_{KL}(q\|m)}, \quad m = \frac{p+q}{2}$$

Bounded in $[0, 1]$ under base-2 logs; 0 means identical. An empty observed window returns `None` – excluded, never scored as zero.

Aggregation (`scoring.py`, `algorithm porygon.distance.v1`, fixed and versioned):

$$\text{score} = 0.50 \cdot \overline{D_{JS}}_{6 \text{ families}} + 0.30 \cdot \text{novelty} + 0.20 \cdot \bar{z}_5 \text{ numeric}$$

Categorical family weights: executable 0.25, parent–child 0.20, sequence bigram 0.20, process name 0.15, user UID 0.10, runtime action 0.10. A missing family is excluded and the remaining weights are renormalized – never imputed as zero.

Study threshold: $D_{JS} \geq 0.25$ is a positive call (`DEFAULT_THRESHOLD`, the `baseline_like` band edge).

Exploratory alternative, used for no reported number. `calibrated_rarity.py` implements an unweighted-mean fusion over four rarity components with a conformal-style exceedance p -value:

$$p = \frac{1 + \#\{\text{calibration values} \geq \text{test}\}}{n + 1}$$

It is feature-flagged off (`calibrated_enabled`, default `False`); the 216-trial study used the production JS-distance path, not this module.

Not a probability. A JS-distance or fused rarity value is a distributional-deviation measure. Neither module defines or claims a $P(\text{attack})$ field, and `docs/CLAIMS_V1.md` explicitly prohibits reporting one.

Disclosed limitation: the 0.50/0.30/0.20 top-level weights are a fixed implementation decision, not fit to data. This is stated rather than hidden, and is why the unweighted alternative exists in the codebase at all.

1. Methodology — Dataset, Splits, and Evaluation Metrics

Rubric: Datasets, Preprocessing & Evaluation Metrics (5 Marks)

No public dataset answers this question. Data is generated from three real OCI images, each pinned by frozen index digest, not a resolvable tag:

Workload	Pinned tag
WL-NGX-V1	nginx:1.26.3-alpine
WL-RDS-V1	redis:7.2.7-alpine
WL-PG-V1	postgres:16.6-alpine

Digests listed in docs/PROFILE_SCOPE_EXPERIMENT_V1.md.

Two simulated attack scenarios carry planted ground truth: SCN-LOG4SHELL-SIM (CVE-2021-44228-shaped spawn) and SCN-RUNC-ESCAPE-SIM (CVE-2019-5736-shaped write), both harmless, plus SCN-EXEC, a capture canary, not ground truth.

Independent unit: the whole workload run. Fit / calibration / test splits are assigned per run by a deterministic run-ID hash before any outcome is observed (artifacts.py:assign_split), so adjacent near-duplicate windows cannot straddle a split.

Primary metrics (docs/RESEARCH_PROTOCOL_V1.md)

Metric ID	Definition
MET-FPR-001	benign runs with ≥ 1 positive call; Clopper–Pearson interval
MET-REC-001	ground-truthed runs detected within the frozen horizon
MET-OTP-001	occurrence-to-persistence latency (declared; not reported this round)
MET-CPU-001	CPU-seconds, RSS, disk writes per workload-minute
MET-CAL-001	held-out benign coverage vs. the calibrated threshold

Realised split, pilot run (not yet confirmatory): of 216 real container trials, $n = 53$ held-out benign runs feed MET-FPR-001 and $n = 96$ ground-truthed scenario runs feed MET-REC-001.

Reporting standard: every percentage ships with numerator, denominator, and a 95% confidence interval – no bare accuracy figures.

2. Implementation Progress & Module Functionality

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

- **Implementation Status Overview:**

Module / Component	Current Implementation State	Status
Kernel Telemetry & Ingestion (Falco eBPF, Docker Collector, Telemetry Adapter)	Captures container process execution via eBPF; durable SQLite outbox with retry	Functional
Behavioural Baseline & JSD Scoring (baseline.py, scoring.py)	Digest-bound behavioural baselines; JSD-based anomaly distance scoring	Functional
Deterministic Detection Engine (detection.py)	7 correlation rules (POR-DET-001-007) generating findings/incidents	Functional
4-Arm Scope Comparison (experiments/)	216-trial pilot run analysed end to end, exact statistical pipeline (not yet confirmatory-grade)	Complete

2. Implementation Progress (contd.)

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module / Component	Current Implementation State	Status
Vulnerability & SBOM Pipeline (Scanner, Trivy 0.72.0)	CycloneDX SBOM generation; EPSS & CISA KEV enrichment	Functional
Response Policy Engine (response.py)	Severity/confidence-gated containment recommendations; human-approval required	Functional
Module Integration	8 containerized services integrated via Docker Compose with network segregation	Integrated

Core Functionality Demonstration: A real, functional XMRig cryptominer binary from a published malware-sample container image was executed against a live Porygon baseline built from 243 real Falco-captured events, with zero prior knowledge of the binary given to the system. Resulting anomaly score: 0.2696 (elevated band, threshold 0.25), with the executable and its parent-child call edge both flagged as previously unseen. Full record: docs/LIVE_DEMO_RECORD_V1.md.

Live Test Results and Comparison

Pilot statistical run (216 trials, not yet confirmatory) alongside the live malware demonstration

Baseline arm	FPR (benign, n=53)	Recall (attacks, n=96)
ARM-GLOBAL (pooled, one baseline for all)	100% [93.3–100%]	100%
ARM-TAG (per mutable image tag)	0% [0–6.7%]	100%
ARM-DIGEST (per exact image digest)	0% [0–6.7%]	100%
ARM-CONTEXT (digest + runtime config)	0% [0–6.7%]	100%

CONTEXT vs. GLOBAL: $p = 6.7 \times 10^{-16}$ (significant). CONTEXT vs. DIGEST, the harder comparison this project's thesis rests on: $p = 1.0$ (null, arms agree on all 53 held-out trials – reported as-is).

Independent live test (not part of the 216-trial protocol): a real XMRig cryptominer binary, from a published malware-testing container image, executed against a Porygon baseline built from 243 genuinely captured Falco events, scorer given no prior knowledge of the binary.

- Anomaly scorer: flagged, score 0.2696 (elevated band, threshold 0.25) – correct detection with zero prior signature.
- Deterministic rule POR-DET-004: did *not* fire – xmrigh is absent from its hardcoded dual-use-tool list, a real, disclosed gap.

Full record: docs/LIVE_DEMO_RECORD_V1.md.

Deterministic Detection Rules (detection.py)

Seven fixed rules producing explainable findings

Rule	Trigger condition	Sev.	Conf.	Inc.
POR-DET-001	Completed window scores ≥ 0.50 behavioural distance	0.65	0.70	No
POR-DET-002	Shell executable absent from the digest baseline executed	0.72	0.90	Yes
POR-DET-003	UID 0 process executed with UID 0 absent from baseline	0.78	0.90	Yes
POR-DET-004	Downloader/network/interpreter/encoding tool absent from baseline	0.64	0.80	Yes
POR-DET-005	Unseen shell followed by unseen dual-use tool, same window	0.90	0.95	Yes
POR-DET-006	Docker exec activity during the observation window	0.35	0.95	No
POR-DET-007	Container reported privileged mode during the window	0.92	0.95	Yes

An incident requires at least one *incident-eligible* finding; POR-DET-001 and POR-DET-006 alone never open an incident, they only contribute evidence. Severity/confidence weights are fixed constants in source, not tuned per deployment.

3. Technical Knowledge & Design Decisions

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Distance metric selection: Jensen-Shannon divergence (JSD) over Kullback-Leibler (KL) divergence. KL divergence is asymmetric ($D_{KL}(P, Q) \neq D_{KL}(Q, P)$) and unbounded above, so a KL score computed for one container cannot be compared against a KL score from a different container on a common scale. JSD is the symmetric, bounded variant ($JSD(P, Q) = JSD(Q, P)$, range $[0, 1]$), which permits one fixed threshold to apply uniformly across all containers.

Containment decision logic. A finding does not trigger any action unless the rule engine's severity (*Sev*) and confidence (*Conf*) scores clear a defined threshold. Pause requires $Sev \geq 0.70$ and $Conf \geq 0.55$. Stop requires $Sev \geq 0.90$, $Conf \geq 0.75$, and a match on one of two specific high-confidence rules (unseen-shell-to-tool sequence, or a container transitioning to privileged mode). Every action, at either threshold, requires explicit human operator approval before execution; no threshold triggers automatic execution.

Statistical significance of the primary result. On 216 real container trials, per-image baseline scoping reduced the false-alarm rate from 100% to 0% relative to a single pooled baseline, with zero reduction in attack detection. The exact statistical test applied to this result returns $p = 6.7 \times 10^{-16}$, several orders of magnitude below the conventional $p < 0.05$ significance threshold.

3. Technical Knowledge & Design Decisions (contd.)

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Multicall binary handling. Utilities such as BusyBox and Toybox implement dozens of commands inside a single executable file; the command executed depends on the invocation name, not the file path. Tracking file identity alone would classify every invoked command as the same previously-seen program. The detection engine instead tracks the invocation (command) name, so a new command invoked through an already-known multicall binary is still flagged as novel.

Canonical scan identity. Each vulnerability scan result is keyed by a SHA-256 hash of six inputs: image digest, local image ID, scanner name, scanner version, vulnerability database schema version, and a researcher-supplied scan reference. Re-executing a scan with an identical set of these six inputs returns the existing stored result (idempotent). A change to any one input produces a distinct, separately-tracked scan record (immutable).

Network segregation. The eight services are partitioned across three isolated Docker networks: ingress (external traffic), internal (inter-service communication), and egress (outbound-only, used exclusively by the vulnerability scanner). A service is not granted network reachability to segments outside its assigned role.

Versioned scoring configuration. The scoring algorithm and its numeric weights are bound to a named algorithm version. A weight modification requires incrementing the version identifier; no weight change is applied in place. Every historical score therefore remains reproducible against the exact configuration that produced it.

3. Technical Knowledge & Design Decisions (contd.)

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

- **Confirmed Findings (216 real trials, exact statistical pipeline):**

- Per-image scoping vs. pooled baseline: significant, $p = 6.7 \times 10^{-16}$, FPR 100% $\rightarrow$ 0%, recall unchanged (96/96 both arms).
- Digest-plus-context vs. digest-only: no measurable difference in this dataset ($p = 1.0$, expected here since the two arms produced identical confusion-matrix counts – not a reporting error), reproduced at both $n = 35$ and $n = 53$ – traced to the tested configuration change (a dropped Linux capability) not altering which processes execute, which the process-name feature cannot see by construction.
- Follow-on feature scoring the security-configuration document directly separates every tested case cleanly: 0/26 false positives, 27/27 correct detections.

Adversarial Scenario Analysis

18 scenarios classified against the actual implementation, not asserted

- **Structurally invisible (6 of 18):** attacks reusing an already-baselined process, adding no new `execve` event – C2 domain repoint, exfiltration over an existing channel, file-only webshell, DNS tunnelling, fileless execution, payload inside an already-baselined HTTPS request. No syscall-argument or network telemetry exists in v1; explicitly out of scope in `THREAT_MODEL_V1.md`, not a hidden gap.
- **Depends on process-identity granularity (4 of 18):** in-place binary replacement keeping the same path/name is likely invisible (identity tracked by name/path, not content hash); SUID privilege escalation to UID 0 *is* caught (POR-DET-003).
- **Untested failure modes (3 of 18):** baseline poisoning during the fit window (defended by design, but the human-review step itself never exercised); low-and-slow attacks beyond the 120s correlation window; process-sequence-bigram mimicry. All three named directly in the project's own threat model.
- **Where the design genuinely helps (5 of 18):** backdoored entrypt script (POR-DET-002); manual docker exec (POR-DET-006); flip to `-privileged` (POR-DET-007); mutable-tag digest substitution (digest scoping only); classic `curl | bash` dropper pattern (POR-DET-005, confidence 0.95).

Full scenario-by-scenario table: `docs/ADVERSARIAL_SCENARIOS_V1.md`.

Verification and Testing Rigor

How correctness of the claimed results is actually checked

- **Layered verification** (`scripts/verify_all.sh`): static (lint/type checks), unit (210 backend and service test functions), live-safe (non-disruptive real Docker container checks), scanner-live/experiment-live (pinned-digest real-container acceptance tests) – separate make targets, none mocked end-to-end.
- **Frozen protocol before results**: the statistical protocol (`porygon.research.protocol.v1`) was written and reviewed *before* the pilot run, with a pre-registered sample size, primary contrast, and material-effect threshold, marked FROZEN – cannot be edited post hoc to fit a result.
- **Six real bugs found and fixed** to obtain the first pilot result: a wrong API field name silently blocking all detections, dead code blocking attack scenarios, a fit-reference contamination bug, and an infinite-recursion bug in the Clopper–Pearson confidence-interval code itself. Commit range in `CONFIRMATORY_RESULT_V1.md`.
- **Reproducibility**: every reported number has a named artifact path and a documented reproduce command.
- **Disclosed limitation**: single-reviewer self-review, not independent third-party peer review – stated plainly.

3. Technical Knowledge — Trade-offs

Rubric: Design Choices & Trade-offs (5 Marks)

Benefit accepted	Cost accepted
Low-overhead process-execution signal, no app instrumentation	Zero network, DNS, socket, or file-access telemetry
Finer-grained baselines per digest and context	Fewer runs per stratum; some return <code>insufficient_profile</code>
Fixed, disclosed scoring weights	Not claimed to be the most accurate achievable weighting
Exact intervals (Clopper–Pearson, McNemar)	More runs per arm than a point estimate needs
Versioned distance-to-score mapping	Valid only while expected behaviour does not itself drift
No automated disruptive response	Slower operator reaction to a confirmed incident
One language (Python) across all services	A known single-process write-throughput ceiling

Edge cases handled in code, with tests

- Backend outage → SQLite outbox spool, exponential backoff retry
- Sensor restart → durable telemetry cursor, no rewind
- Duplicate delivery → deterministic event ID, dedup upsert
- Malformed input → rejected, kept as a short scrubbed sample
- Too little data → `insufficient_profile`, never a guessed score
- Runtime-context fingerprint redacts secret-shaped flags and values; the raw process `command_line` is stored as-is, *not* redacted – a disclosed residual exposure

3. Technical Knowledge — Claim Boundaries

Rubric: Preliminary Observations (5 Marks) — docs/CLAIMS_V1.md governs what may be stated

What is, and is not, yet claimed

Claim	Status at this review
Per-image scoping beats a pooled baseline	Supported , $p=6.7 \times 10^{-16}$ (216 trials)
Digest-plus-context beats digest-only	Null , $p=1.0$, reproduced at $n=35$ and $n=53$
Direct context-delta feature works	Exploratory only ; hand-fixed weights, no attack scenarios
Recall gate properly stress-tested	No – recall was 100% with zero variance in every arm
Suitable for production	Not claimed

The 216-trial re-run strengthened the two FPR contrasts, which had real measurable outcomes. It did *not* strengthen the recall gate: with zero recall variance, there was no difference for the test to detect.

Honest process notes

- 1 An early draft framed the easy win (per-image vs. pooled) as though it supported the harder thesis (context vs. digest). Caught and corrected in self-review before publication; the null is now reported as the outcome of the actual thesis contrast.
- 2 Single-reviewer self-review for both security and methodology, not independent peer review.

Unusual is not the same as malicious.

- Unusual but harmless: scheduled backups, operator debugging, restarts, a traffic spike.
- Harmful but ordinary-looking: an attacker using only tools already present in the baseline.

An anomaly score is deviation evidence, never a probability of compromise.

Team Contribution and Project Status

Phase 2 deliverables against the original scope

Deliverable	State at this review
eBPF/Falco telemetry pipeline	Functional, integrated, durable outbox delivery
Baseline + JSD scoring engine	Functional, versioned configuration
7-rule deterministic detection	Functional, POR-DET-001-007
Vulnerability/SBOM enrichment	Functional, Trivy + EPSS + CISA KEV
Response policy engine	Functional, human-approval gated
4-arm scoping pilot study (not yet confirmatory)	Complete, 216 trials, two reproduced results
Live real-malware demonstration	Complete, XMRig, documented including the miss
Independent literature verification	Complete, 10 papers checked by DOI/API/direct fetch

All eight modules are integrated into one running Docker Compose stack (`make up`) and independently exercised through `scripts/verify_all.sh`, not demonstrated in isolation.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- **Individual MOOC / Online Certification Selection:**

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Ashin Varghese	Introduction to Kubernetes (LFS158)	edX	15 Hrs
2	Anurup Krishna	Introduction to Kubernetes (LFS158)	edX	20 Hrs
3	Krishnan S	Introduction to Kubernetes (LFS158)	edX	12 Hrs
4	Madhav Sreejith	Introduction to Kubernetes (LFS158)	edX	16 Hrs

- **Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.

Limitations & Future Work

Limitations:

- The pilot 216-trial run (not yet confirmatory) covers the tested attack and workload set; it does not claim coverage of every real-world attack pattern or benign workload shape.
- Scoring uses categorical process-name/edge features; it does not yet model syscall argument content or network payloads.
- Response actions require a human operator in the loop by design – there is no fully autonomous containment path.

Future Work:

- Extend the per-image + configuration baseline to a larger, more diverse container corpus beyond the 216-trial dataset.
- Evaluate additional distance metrics and richer feature sets (e.g. syscall arguments) against the same statistical protocol.
- Explore semi-automated response for the highest-confidence rule matches, still gated by policy and audit logging.

References I



G. R. Castanhel, T. Heinrich, F. Ceschin, and C. A. Maziero, "Taking a peek: an evaluation of anomaly detection using system calls for containers," in *Proc. IEEE Symp. Computers and Communications (ISCC)*, 2021.



A. El Khairi, M. Caselli, C. Knierim, A. Peter, and A. Continella, "Contextualizing system calls in containers for anomaly-based intrusion detection," in *Proc. ACM Cloud Computing Security Workshop (CCSW)*, 2022, pp. 9–21.



M. A. Bouke *et al.*, "Multi-level distributional entropy for explainable network intrusion detection," arXiv:2606.29797, 2026.



G. Fournier, S. Afchain, and S. Baubeau, "Runtime security monitoring with eBPF," in *Proc. Symp. sur la Sécurité des Technologies de l'Information et des Communications (SSTIC)*, 2021.



X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, "eBPF-Guard: a detection method for container escape via multi-level monitoring and enhanced analysis model," *Empirical Software Engineering*, vol. 31, article 51, 2026.



Li Wei, Yuan Zekun, Wu Kehe, and Cheng Rui, "Container anomaly detection based on attention mechanism and multiscale convolutional neural network," *Journal of Information Security Research*, vol. 11, no. 1, pp. 35–, 2025.



K. Xiong, Z. Wu, and X. Jia, "DeepContainer: a deep learning-based framework for real-time anomaly detection in cloud-native container environments," *Journal of Advanced Computing Systems*, 2025.



J. H. Ring, C. M. Van Oort, S. Durst, V. White, J. P. Near, and C. Skalka, "Methods for host-based intrusion detection with deep learning," *Digital Threats: Research and Practice*, vol. 2, no. 4, 2021.

References II



The Falco Authors, “Falco: cloud-native runtime security,” Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, “OCI image format specification.” [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You